PCCCS495: Advanced Programming (OOP) Lab

Term-II Individual Project

Title: STUDENT ATTENDANCE MANAGEMENT SYSTEM

Submitted by-

Zeba Afreen

Roll No: 77

Dept: IOTCSBCT

Enrolment no: 12024052017078

Instructor Name:
Dr. Swagatam Basu, Dr. Susovan Jana, Dr. Soumadip Biswas

Spring 2026

2. Problem Statement

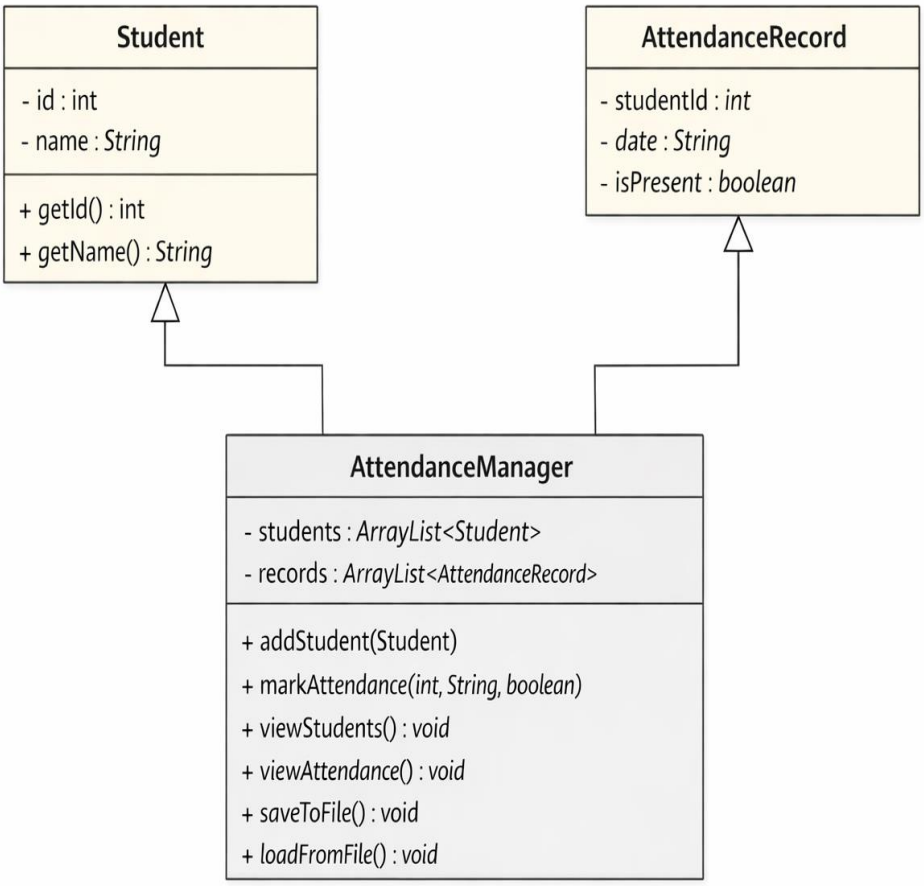
Managing student attendance manually is inefficient and prone to errors, especially when dealing with multiple students and dates. Existing manual systems lack proper organization and are difficult to maintain over time.

This project aims to develop a Student Attendance Management System that allows storing student data, recording attendance, and retrieving it efficiently. The system ensures proper validation of inputs and avoids invalid operations such as marking attendance for non-existent students. It also supports file handling to persist data.

The focus of this project is not just functionality, but clean object-oriented design using Java

3. System Design

3.1 UML Class Diagram



3.2 Description of Major Classes

- **Student**: Stores student ID and name
- **AttendanceRecord**: Stores attendance data for a specific date
- **AttendanceManager**: Handles all operations like adding students, marking attendance, and file handling

3.3 Package Structure

```
src/
├── main/
├── model/
└── service/
```

3.4 Interaction Flow

1. User runs the program
2. Students are added
3. Attendance is marked
4. Data is stored in memory
5. Data is saved to file
6. Data can be loaded back

4. OOP Concepts Used

4.1 Encapsulation

Where used: In Student and AttendanceRecord classes
Why used: To protect data and allow controlled access

Code:

```
private int id;

public int getId() {
    return id;
}
```

Explanation:
The variable id is private, meaning it cannot be accessed directly. It can only be accessed through the getter method, ensuring controlled access.

4.2 Abstraction

Where used: In AttendanceManager methods
Why used: To hide internal logic from the user

Code:

```
manager.markAttendance(1, "2026-03-18", true);
```

Explanation:
The user does not need to know how attendance is stored internally. The method provides a simple interface.

4.3 Inheritance

Where used: (If you added Person → Student, mention here. Otherwise keep simple)

Explanation:
Inheritance allows code reuse and hierarchical design.

4.4 Polymorphism

Where used: toString() method in AttendanceRecord

Code:

```
@Override
public String toString() {
    return "Student ID: " + studentId + ", Date: " + date;
}
```

Explanation:
The method overrides default behavior to provide custom output.

4.5 Exception Handling

Where used: File handling

Code:

```
try {
    BufferedWriter writer = new BufferedWriter(new FileWriter("file.txt"));
} catch (IOException e) {
```

```
    System.out.println("Error: " + e.getMessage());
}
```

Explanation:
Prevents program crash and handles runtime errors gracefully.

4.6 Collections

Where used: ArrayList

Code:

```
ArrayList<Student> students = new ArrayList<>();
```

Explanation:
Used to store dynamic data efficiently.

5. Implementation Highlights

Snippet 1: Adding Student

```
public void addStudent(Student student) {
    students.add(student);
}
```

Justification:
Simple method to store student data dynamically.

Snippet 2: Attendance Validation

```
if (!studentExists(studentId)) {
    System.out.println("Error: Student not found");
    return;
}
```

Justification:
Prevents invalid data entry.

Snippet 3: File Saving

```
writer.write(r.getStudentId() + "," + r.getDate());
```

Justification:
Stores data in structured format for persistence.

6. Testing and Edge Cases

Test Cases

Scenario	Expected Result
Add student	Student added successfully
Mark attendance	Record stored correctly
View students	List displayed

Edge Cases

- Invalid student ID
- No students in the system
- Duplicate student entries
- Empty attendance records

These cases are handled to ensure smooth execution of the program.

Failure Scenarios

- File Not Found
- Invalid Student ID
- Empty Data Handling
- Incorrect File Format

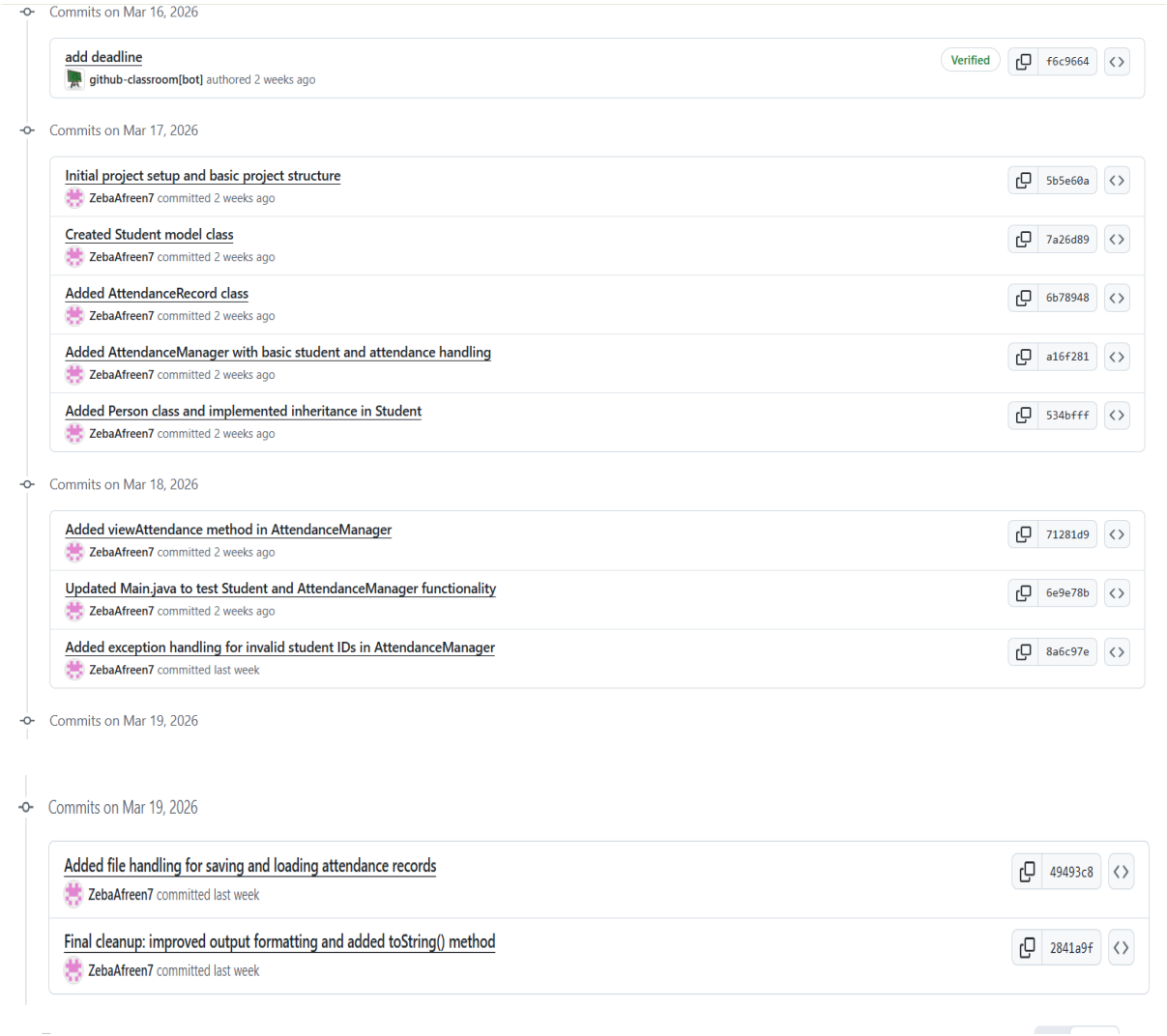
7. Git Workflow

The project was developed incrementally using version control on GitHub. Multiple commits were made to show progress over time.

Examples of commits:

- Initial project setup
- Added Student class
- Added AttendanceRecord class
- Implemented AttendanceManager
- Added inheritance using Person class
- Improved attendance handling

This demonstrates a clear and structured development process.



8. Conclusion & Future Scope

The project successfully demonstrates the use of object-oriented programming concepts in Java. It provides a simple and effective solution for managing student attendance. The modular design makes it easy to understand and extend.

- Add graphical user interface (GUI)
- Connect to a database
- Implement login system
- Generate attendance reports

Appendix

Initial project proposal:

Title: **Student Attendance Management System**

Problem Statement: In many schools and colleges, attendance is still recorded manually or stored in simple spreadsheets. This can make it difficult to maintain accurate records and quickly check attendance history for students. Teachers may also find it time-consuming to calculate attendance percentages or review records for multiple students. A Student Attendance Management System can help organize this information in a more structured way. The proposed system will allow teachers to add students, mark attendance for specific dates, and view attendance records easily. The system will store attendance data so that it can be accessed later when needed. This project will be implemented using Java and will demonstrate object-oriented programming concepts such as encapsulation, inheritance, and modular class design.

Target User: Teachers or instructors who need to record and check student attendance.

Core Features:

- Add and manage student information
- Mark attendance for a specific date
- View attendance records of students
- Calculate attendance percentage
- Save and load records using files
- Basic error handling for invalid input

OOP Concepts to be Used:

Abstraction: Classes such as *Student* and *AttendanceRecord* represent real-world entities and hide internal details.

Inheritance: A base *User* class may be extended by *Student* or *Teacher*.

Polymorphism: Methods like *displayDetails()* can be implemented differently in subclasses.

Exception Handling: Used for invalid input, missing student IDs, or file-related errors.

Collections: *ArrayList* will store student records and attendance data.

Architecture: The system will use a simple object-oriented structure with classes such as *Student*, *AttendanceRecord*, and *AttendanceManager*. The manager class will handle operations like adding students, marking attendance, and retrieving records. Java collections will store the data during program execution, and file handling will be used to save and load records so that data is not lost when the program closes.

Review mail received

Dear Zeba Afreen,

Project: Student Attendance Management System
Decision: APPROVED MINOR
Score: 85

Strengths:
The proposal clearly identifies a common problem in educational settings and offers a practical, structured solution. The target users and core features are well-defined and appropriate for a university Java OOP project. The basic architectural classes (Student, AttendanceRecord, AttendanceManager) are sensible, and the inclusion of file-based persistence and error handling demonstrates a practical understanding of application development. The explicit mention of key OOP concepts, collections, and modular design is a strong point.

Improvements:
While the proposal mentions demonstrating 'inheritance' and 'polymorphism', the current feature set and architectural description do not immediately provide clear use cases for these concepts. Consider adding a feature or refining the architecture to naturally incorporate these, for example:
1. **Inheritance:** Introduce different types of students (e.g., 'UndergraduateStudent', 'PostgraduateStudent') that inherit from a base 'Student' class, each with potentially unique attributes or methods.
2. **Polymorphism:** If different student types are introduced, demonstrate polymorphic behavior when processing attendance for various student types, or consider different types of 'AttendanceEvents' (e.g., 'Present', 'Absent', 'Excused') that might be handled polymorphically.
This would strengthen the demonstration of all promised OOP concepts.

Regards,
Course Instructor